
OpenWAM

User Guide

Building and running one-dimensional engine and gas-dynamics models
with WAMer and the OpenWAM solver

For the current build (RoeM interior + GJM junctions)

CMT — Universitat Politècnica de València

This guide documents the current version only. Where an older tutorial differs in units, screens, or procedure, this guide takes precedence.

Contents

1	Introduction	4
1.1	Two programs	4
1.2	Who this guide is for	4
1.3	Two rules that apply everywhere	4
2	Getting started	6
2.1	The WAMer window	6
2.2	The mental model	6
2.3	What a finished model looks like	6
3	Building a model	9
3.1	The two invisible mandatory nodes	9
3.2	Placing and connecting components	9
3.3	The component palette	9
3.4	Boundaries: how a model meets the outside world	11
3.5	Species and composition	11
3.6	Wall temperature and heat transfer	11
3.7	A note on stability, corrected	11
3.8	A worked reference	11
4	Engine subsystem recipes	13
4.1	Four-stroke spark-ignition engine	13
4.2	Two-stroke engine	13
4.3	Combustion choices	13
4.4	Intercooler	16
4.5	Exhaust gas recirculation (EGR)	16
4.6	Vehicle and transient operation	16
5	Turbocharging	17
5.1	Topology	17
5.2	Maps	17
5.3	Twin-scroll turbine	17
5.4	Variable geometry (VGT)	17
5.5	Wastegate: mechanical or controlled	18
6	Control systems	19
6.1	Wiring rules	19
6.2	What can be controlled	19
6.3	The PID controller	19
6.4	The 1D table	20
6.5	The mechanical wastegate	20
6.6	External co-simulation does not work	20

7	Choosing what the solver writes	22
7.1	Average and instantaneous results	22
7.2	Run duration	22
7.3	Space-time (wave) results	22
7.4	Parametric study (batch sweep)	22
8	Running a simulation	25
8.1	The correct flow	25
8.2	Other ways to run	25
8.3	Batch runs (.gro)	25
9	Viewing and post-processing results	26
9.1	The quickest path	26
9.2	WAMer's plotter	26
9.3	Building a trimmed results file	26
9.4	An empty graph window	26
10	Stability, errors, and troubleshooting	28
10.1	Instability triggers, by solver method	28
10.2	Solver methods	28
10.3	Crash checklist	28
10.4	Duration and convergence	28
11	Reference and cross-links	29
11.1	Companion documents	29
11.2	The original tutorials	29
11.3	File types	30

List of Figures

2.1	The Cascade (top) and Mosaic (bottom) window arrangements.	7
2.2	A complete naturally aspirated model (Mitsubishi 3B21) laid out on the WAMer canvas.	7
2.3	The turbocharged version (3B21T). Note the green control-logic lines threading the boost-control chain and the red “C” on the controlled wastegate — both explained in Chapter 6.	8
3.1	The Execution (Ejec) node: ambient conditions, numerical method, species model, and output setup for the whole run.	10
3.2	A duct’s data: mesh, duct type, initial gas and wall state, geometry, and per-duct numerical method.	10
4.1	The engine block: working conditions, geometry, heat transfer, mechanical losses, and combustion.	14
4.2	A poppet valve: reference diameter, lift table (.LEV), and flow-coefficient table (.FLU).	15
6.1	A sensor: measured quantity, time constant, gain, and position. Gain 0 silently disables it.	20
6.2	The PID controller: positive/negative gains, output limits, period, and setpoint. . .	21
6.3	The 1D table: period, interpolation mode, and either inline data or a .T1D file. . .	21
7.1	Create Parametric Study: the per-case value grid.	23
7.2	Naming the per-case project and output files, and the batch to run.	24
9.1	The results-visualization window: the <i>X/Y</i> variable pickers, the add-curve arrow, and the multi-.DAT controls.	27
9.2	A plot from the WAMer plotter: wastegate position against time for the 3B21T at 3250 rpm, hunting up and down and damping toward its steady opening as the run converges.	27

Chapter 1

Introduction

OpenWAM is a free, open-source, one-dimensional unsteady compressible gas-dynamics code for internal combustion engines and their intake and exhaust systems. It represents ducts as quasi-one-dimensional elements in which pressure waves travel along the axis, cavities (plenums, cylinders, turbine and compressor volumes) as zero-dimensional elements in which the gas state is uniform, and the connections between them with non-dimensional boundary models. It solves the thermodynamics and fluid dynamics of the whole engine breathing system — intake, cylinders, exhaust, and turbocharger — as one coupled network. For the theoretical background (heat-release laws, the discharge coefficient, the treatment of λ and combustion) see J. M. Pérez López's study of OpenWAM ([Original Tutorials/JOSE-MARIA-Thesis.pdf](#)); this guide does not restate it.

1.1 Two programs

Working with OpenWAM means using two separate executables:

- **WAMer** (WAMer.exe) — the graphical editor. You draw the engine as a flowchart of components, fill in their fields, and save the model as a .WAM text file. WAMer also keeps a binary project file, .PCS, holding editor-only information (canvas layout, parametric-study definitions) that the solver never reads.
- **OpenWAM** (OpenWAM.exe) — the solver. It is a console program that reads one .WAM file and writes the results as tab-separated .DAT files.

Nothing needs to be installed beyond having the two executables. WAMer builds and launches the solver for you, or you can run the solver directly on a .WAM file (Chapter 8).

1.2 Who this guide is for

This guide is a practical manual for building, running, and post-processing engine models. It assumes familiarity with reciprocating-engine terminology but not with OpenWAM. It documents workflows; the companion *OpenWAM Glossary* documents every field of every component with its meaning and a sensible default, and the *OpenWAM Developer Guide* maps each behaviour to the source file that implements it. Where a field is only named here, the Glossary carries its full definition.

1.3 Two rules that apply everywhere

- **Use decimal points, never commas.** Older tutorials are inconsistent about this; the current version expects a point as the decimal separator in every numeric field.

-
- **This guide is for the current version only.** Ignore units, screens, or procedures in older tutorials that differ from what is described here.

Chapter 2

Getting started

2.1 The WAMer window

WAMer opens on a blank flowchart canvas. Along the top are the menu bar and toolbar; a component palette, organised into tabs (Chapter 3), supplies the elements you place on the canvas; a data panel shows the fields of the currently selected component. The basic UI walkthrough is covered by the “Exemple d’utilisation” tutorial ([Original Tutorials/Exemple-d'utilisation-EN.pdf](#)) — this guide does not repeat the click-by-click tour, only the parts that have changed or that cause confusion.

Every window you open — a project, an execution-control view, a graph — persists and is reachable from the **Window** menu. That menu also offers **Cascade** (each view becomes an ordinary minimise/maximise/close window inside WAMer) and **Mosaic** (all views tiled at once), shown in Figure 2.1.

2.2 The mental model

Building and running a model is always the same sequence:

1. place components on the canvas;
2. connect them with ducts (a duct *is* the connector — see Chapter 3);
3. fill in each component’s fields;
4. add the two invisible mandatory nodes (Execution, and Engine for engine models);
5. choose which results the solver should write;
6. run the solver;
7. open the .DAT results.

2.3 What a finished model looks like

It helps to keep a finished model in view while the pieces below are introduced. Figure 2.2 is a complete naturally aspirated single-cylinder engine; Figure 2.3 is its turbocharged counterpart, with the compressor/intercooler intake path, the turbine and wastegate exhaust path, and the boost-control wiring all visible. Refer back to these as each component type is described.

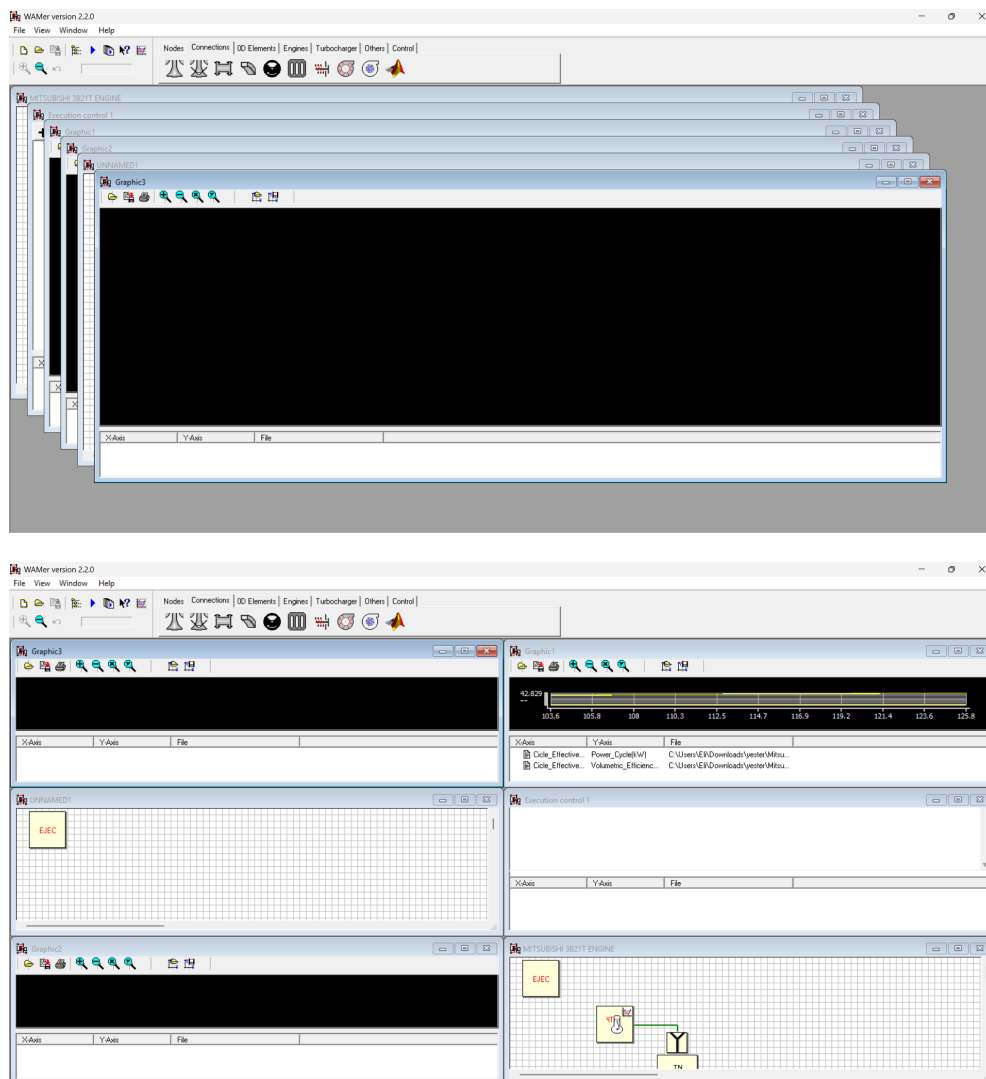


Figure 2.1. The **Cascade** (top) and **Mosaic** (bottom) window arrangements.

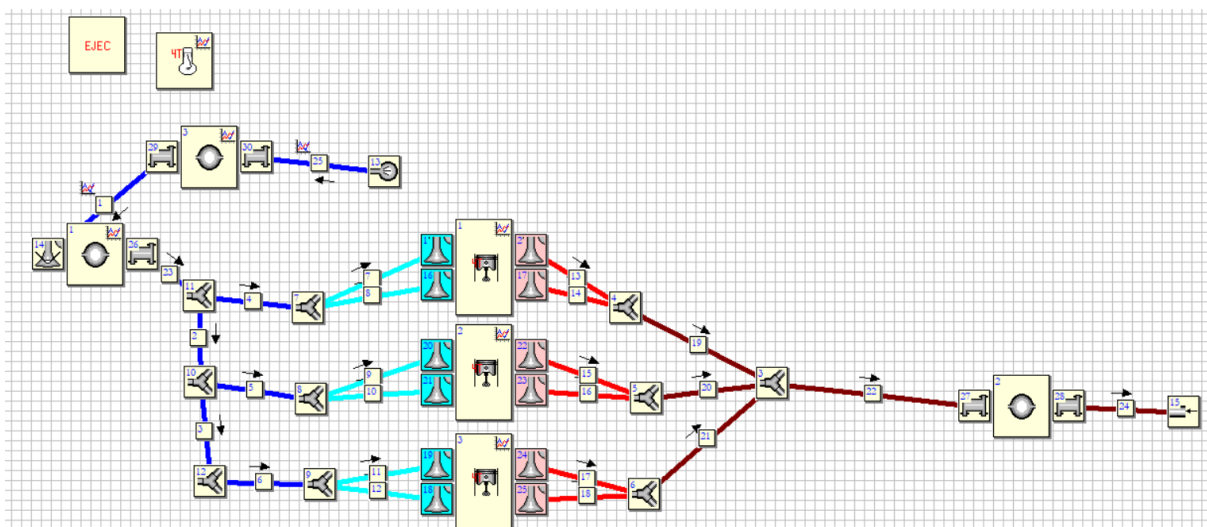


Figure 2.2. A complete naturally aspirated model (Mitsubishi 3B21) laid out on the WAMer canvas.

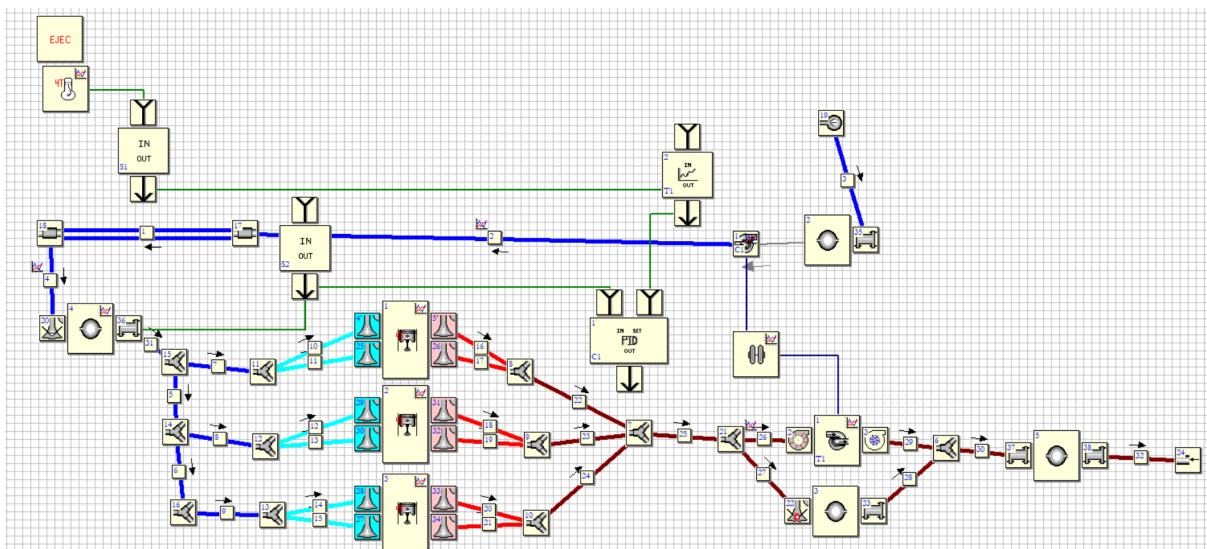


Figure 2.3. The turbocharged version (3B21T). Note the green control-logic lines threading the boost-control chain and the red “C” on the controlled wastegate — both explained in Chapter 6.

Chapter 3

Building a model

3.1 The two invisible mandatory nodes

Every model needs an **Execution (Ejec)** node; every *engine* model additionally needs an **Engine** node. Neither appears on the flowchart canvas — they are configured from menus — but the solver will not run without them. Engine-less rigs (a turbine flow bench, the single-pipe fundamentals cases) have no Engine node, and for them the Execution “duration” is measured in *seconds* rather than engine cycles (Chapter 7). The Execution node also carries the ambient conditions, the numerical method and Courant number, the species and specific-heat-ratio models, and the output settings; the *Glossary*, entry *Execution*, lists every field.

3.2 Placing and connecting components

Drag a component from the palette onto the canvas. **Ducts are the connectors:** dragging from one component to another creates the duct between them. The direction in which you draw a duct defines its positive flow direction, so draw intake ducts in-to-engine and exhaust ducts engine-to-out. A duct is a physical pipe with its own mesh, geometry, wall, and gas state (Figure 3.2); it is not a mere line. The *Glossary*, entry *Duct*, lists its fields.

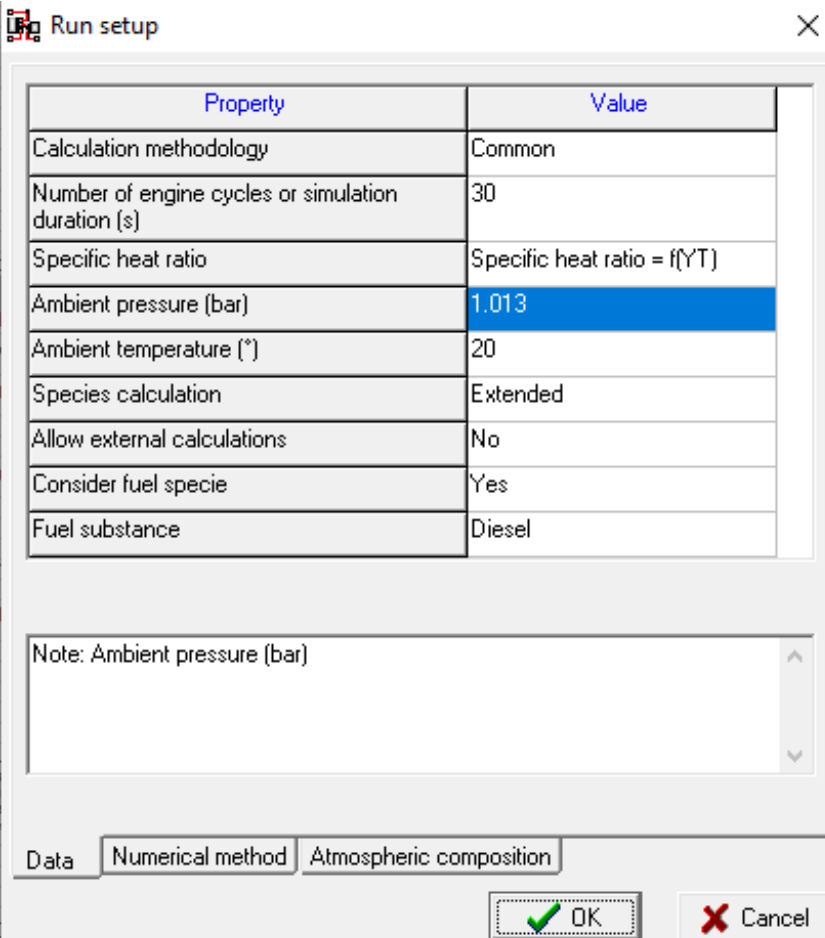
One convenience worth knowing so you do not chase it: each duct’s right-click menu has a **Rigid** entry (default on). It is purely a canvas-drawing property — on, the duct is drawn straight with its midpoint pinned; off, you can drag its centre to route it around other components. It changes nothing in the physics and is not even written to the .WAM.

Ducts never attach directly to a zero-dimensional element. A duct meeting a plenum, chamber, turbine, or compressor volume must pass through a *connection element* — a valve, a throttle, a turbine stator/rotor, or a *Node to Deposit*. A zero-dimensional element with a duct on each side therefore needs *two* connection elements. WAMer refuses to generate the .WAM otherwise.

3.3 The component palette

The palette is organised into tabs. This guide introduces the components by task; the *Glossary* is the exhaustive catalogue, grouped by exactly these tabs.

- **Nodes** — duct-to-duct joints (ramification, union), pressure-loss connectors, and the boundaries that meet the outside world (open/closed/anechoic ends, prescribed inlets, the screw compressor).
- **Connections** — everything that meets a zero-dimensional element: poppet and reed valves, throttle, ports, rotary disk, wastegate, turbine stator/rotor, Node to Deposit.



Run setup

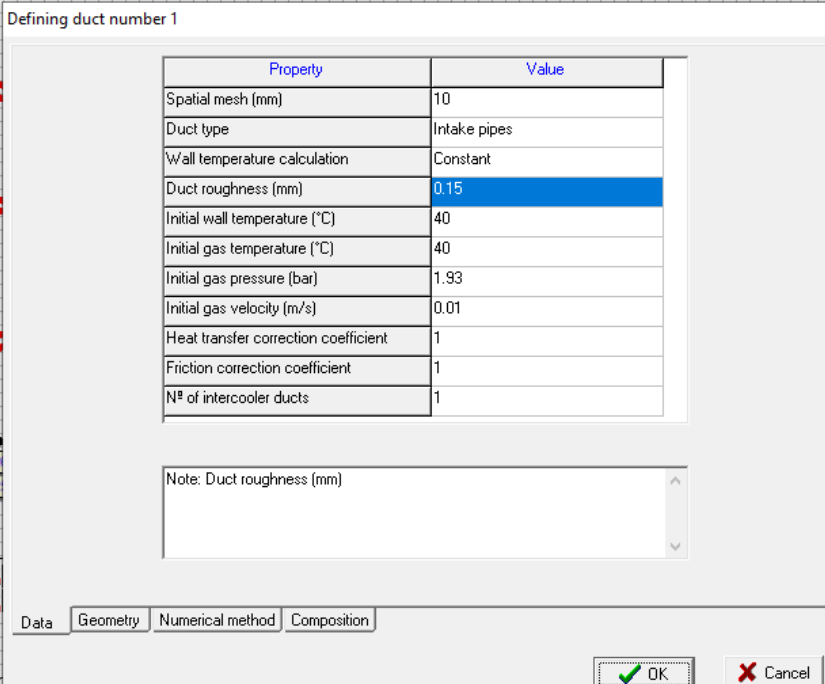
Property	Value
Calculation methodology	Common
Number of engine cycles or simulation duration (s)	30
Specific heat ratio	Specific heat ratio = f(YT)
Ambient pressure (bar)	1.013
Ambient temperature (°)	20
Species calculation	Extended
Allow external calculations	No
Consider fuel specie	Yes
Fuel substance	Diesel

Note: Ambient pressure (bar)

Data Numerical method Atmospheric composition

OK Cancel

Figure 3.1. The Execution (Ejec) node: ambient conditions, numerical method, species model, and output setup for the whole run.



Defining duct number 1

Property	Value
Spatial mesh (mm)	10
Duct type	Intake pipes
Wall temperature calculation	Constant
Duct roughness (mm)	0.15
Initial wall temperature (°C)	40
Initial gas temperature (°C)	40
Initial gas pressure (bar)	1.93
Initial gas velocity (m/s)	0.01
Heat transfer correction coefficient	1
Friction correction coefficient	1
Nº of intercooler ducts	1

Note: Duct roughness (mm)

Data Geometry Numerical method Composition

OK Cancel

Figure 3.2. A duct's data: mesh, duct type, initial gas and wall state, geometry, and per-duct numerical method.

- **0D Elements** — constant- and variable-volume chambers, venturi, directional junction.
- **Engines** — the 4-stroke and 2-stroke engine blocks and their cylinders.
- **Turbocharger** — the shaft, turbine (single and twin-flow), and compressor.
- **Control** — sensors, PID, 1D table, switch, gain.
- **Others** — the Execution node.

3.4 Boundaries: how a model meets the outside world

An intake usually begins at an *Extreme to Atmosphere*; an exhaust ends at an *Extreme with constant P and T* (a reservoir) or at atmosphere. Acoustic work uses *Anechoic* or *Closed* ends. Bench rigs prescribe their inlet with a *Pressure Pulse* or a *Static Pressure Inlet*. Each is a Nodes-tab boundary; the *Glossary* gives their fields and the few traps (for example, the “losses coefficient” on an open end is a flow-openness multiplier where 1 is fully open, not an additive loss).

3.5 Species and composition

Everything with a volume — ducts, chambers, cylinders — has a **Composition** tab. The **simplified** species model tracks three mass fractions (fuel, burnt gas, air); the **extended** model tracks nine (O₂, CO₂, H₂O, HC, Soot, NO_x, CO, Fuel, N₂). The choice is made once, on the Execution node.

ACT combustion requires the extended species model. If you select the predictive ACT diesel combustion model (Chapter 4) while the run is set to simplified species, it will not run correctly.

3.6 Wall temperature and heat transfer

Best practice is to leave all wall temperatures *constant* unless you are modelling an intercooler. There is one duct-type trap that matters here: an intake/exhaust **port** sits in the water-jacketed head, so its wall is pinned to the engine coolant temperature and the Heat Transfer tab is hidden; an intake/exhaust **pipe** is an external runner, so the Heat Transfer tab is shown and the wall sees ambient (or a set coolant temperature). An intercooler must therefore be modelled as a *pipe*, not a *port*, or it cannot shed heat toward ambient (Chapter 4). The *Glossary*, entry *Duct*, spells out the Spanish naming behind this (*pipa* = port, *tubo* = pipe).

3.7 A note on stability, corrected

On the current default solver (RoeM interior, the “TVD” method slot), an initial gas velocity of zero is perfectly stable, and in-pipe area variation (inlet diameter $\neq$ outlet diameter, or a multi-segment duct) is stable at any Courant number, validated from subsonic through supersonic flow. The old warnings that “ $v = 0$ ” or “multiple diameters” cause instability were artifacts of the legacy Lax–Wendroff scheme and apply *only* if you deliberately select Lax–Wendroff+FCT. Do not avoid area variation on the default solver. Chapter 10 covers this in full.

3.8 A worked reference

For a complete naturally aspirated build from an empty canvas, follow the “Exemple d’utilisation” tutorial ([Original Tutorials/Exemple-dutilisation-EN.pdf](#)); the finished model it produces is

the kind shown in Figure 2.2. In outline: place the cylinders and the engine block; add the intake and exhaust valves; draw the intake and exhaust ducts and their manifold plenums (each duct meeting a plenum through a Node to Deposit); terminate the intake at an atmosphere extreme and the exhaust at a reservoir; set the species and ambient conditions on the Execution node; choose the outputs; and run.

Chapter 4

Engine subsystem recipes

4.1 Four-stroke spark-ignition engine

A four-stroke engine is an Engine block plus one cylinder per bore, each cylinder joined to intake and exhaust runners by poppet **Valves**. Each valve carries a **lift table** (.LEV) and a **flow-coefficient table** (.FLU) and an opening angle in the engine's 0–720° frame (0° = firing TDC, 360° = overlap TDC). Combustion is a Wiebe heat-release law. The engine block (Figure 4.1) holds the geometry, the mechanical-loss coefficients, the Woschni heat-transfer coefficients, and the fuelling. Sample .LEV and .FLU files ship in the repository.

“Relative air fuel ratio” is the equivalence ratio φ , not λ . Despite the label, the field is the relative fuel/air ratio $F_r = \varphi = 1/\lambda$. To model a target λ , enter $1/\lambda$ (for example, $\lambda = 1.12$ lean $\rightarrow$ enter 0.89). See the *Glossary*, entry *4T/2T Engine*.

4.2 Two-stroke engine

A two-stroke engine has no poppet valves. Scavenge and exhaust flow through **Ports (Lumbrera)** whose open area varies with piston position; fresh air enters the crankcase through a **Rotary Disk**; the crankcase itself is a **Variable Volume Chamber**; and **Reed Valves** act as one-way petals. The two-stroke example model in the repository is the reference topology. The *Glossary* gives each component's fields.

4.3 Combustion choices

- **Wiebe heat-release law** — for spark ignition and simple diesel. You set the form exponent m , the completeness constant C , the burned fraction β of each sub-law, the burn duration, and the start angle. A single premixed SI burn is one sub-law with $\beta = 1$. The start angle Alpha0 is entered *positive* as degrees before firing TDC (the reader negates it).
- **ACT predictive diesel** — a spray-packet apparent-combustion-time model that also computes NOx. It requires the extended species model, opens two injection tabs, and is compression-ignition only. It is described in the *Glossary*, entry *ACT*.

Injection-tab first-entry bug (ACT). The first time you fill the injection-pulses tab, the UI divides the injection angle by 1000 and the mass percent by 10 before storing. Re-open the tab and re-type the values — that writes them correctly. The bug mangles only the initial entry, so always re-open and verify after first filling it.

Property	Value
Motor type	Compression ignited
Engine operating condition	Steady state
EGR's calculation	No
Number of cycles without thermal inertia	10
Wall temperature calculation	Variable without thermal inertia
Initial engine speed (rpm)	3500
Intake pressure at the intake closing (bar)	1.9

Wall temperature calculation. It is recommended to use variable temperature with thermal inertia for calculations in transitory and without thermal inertia for calculations in stationary.

Working conditions Heat transfer Geometry
Mechanical losses Composition Injection system Injection pulses Control

OK Cancel

Figure 4.1. The engine block: working conditions, geometry, heat transfer, mechanical losses, and combustion.

Defining valve number 5

Property	Value
Valve diameter (mm)	27.4
Valve opening (°)	340
Speed control	Engine
Speed	0
Speed ratio	0.5

Note: valve opening (°)

Data Flow coefficients Lift Control

OK Cancel

Figure 4.2. A poppet valve: reference diameter, lift table (.LEV), and flow-coefficient table (.FLU).

4.4 Intercooler

OpenWAM has no intercooler component. Model one by raising the duct field “**N of intercooler ducts**” (which repeats the duct N times) and setting the wall-temperature mode to shed heat to ambient. The duct *must* be a pipe type, not a port (previous section), or its wall is pinned to the engine coolant and no charge cooling occurs.

4.5 Exhaust gas recirculation (EGR)

There is no dedicated EGR component. Turn on the **EGR flag** in the engine working conditions — this adds a burnt-gas marker species so recirculated exhaust is tracked through the network — and build the loop from ordinary parts: tap an exhaust plenum, run a (cooled) pipe through a throttle acting as the EGR valve, and return it to an intake plenum, each duct meeting a plenum through a Node to Deposit. With ACT combustion, the recirculated burnt gas dilutes the charge and lowers NOx. The flag alone does nothing physical; it only enables the tracking.

4.6 Vehicle and transient operation

Setting the engine operating condition to *Transient variable speed* adds a **Vehicle Model** tab: vehicle, transmission, and wheel masses and inertias; gearbox and final-drive ratios and efficiency; wheel radius; and the rolling, aerodynamic, and grade road loads. Engine speed then evolves from vehicle dynamics each step instead of being fixed. Enable the `Vehicle_Speed` and `Resistant_Torque` outputs to watch a pull. The *Glossary*, entry *4T/2T Engine (Vehicle Model)*, lists every field and notes the one configuration caveat: with no gear-shift model, a single-gear wide-open pull over-revs, so bound the transient or set a road load that balances the drive torque within the engine’s speed range.

Chapter 5

Turbocharging

For the basic compressor + turbine + shaft build and map entry, follow the “Turbocompresseur” tutorial ([Original Tutorials/Turbocompresseur-EN.pdf](#)). This chapter records what to add or correct beyond it. Figure 2.3 shows a complete turbocharged model end to end.

5.1 Topology

The exhaust drives a **Turbine Stator** then a **Turbine Rotor**; the intake is fed by a **Compressor**; both are tied to a **Turbocharger shaft**, which is either fixed-speed or variable-speed with an inertia. The turbine is a zero-dimensional volume, so its exhaust duct reaches it through the stator and its outlet leaves through the rotor; the compressor’s suction attaches to a constant-volume chamber (its inlet volume), whose own inlet duct then needs a Node to Deposit.

5.2 Maps

The compressor reads a .CMP map and the turbine a .TMP map. Prefer the **SAE** compressor format over the legacy WAMer format. Sample maps ship in the repository. The *Glossary* lists the map and reference fields.

Set inlet reservoirs to real conditions. Where a pipe meets a turbine or compressor volume through a Node to Deposit, the UI can default the inlet pressure and temperature to zero, which draws from vacuum and produces a NaN. Set a real 1.013 bar and ambient temperature.

5.3 Twin-scroll turbine

A **Twin Flow Turbine** feeds one rotor from two independent scrolls (two stators, one rotor), keeping the exhaust pulses of different cylinder groups separated for more effective recovery. Its fields are identical to the single turbine; the *Glossary*, entry *Twin-Scroll Turbine*, explains how the single map is shared between the scrolls. Set the initial shaft speed near the expected equilibrium so the turbo does not start off-map.

5.4 Variable geometry (VGT)

A variable-geometry turbine is the *Controlled 2-Nozzle* turbine model driven by a boost-control chain: a boost-pressure sensor feeds a speed-to-boost 1D table (the setpoint schedule) and a PID whose output moves the turbine rack position. The wiring is the subject of Chapter 6.

5.5 Wastegate: mechanical or controlled

Two ways to limit boost:

- **Mechanical wastegate** — a self-actuating spring–mass–damper valve that senses boost on a control duct and opens against a spring. It needs no sensor or controller. Use it when the boost limit is a fixed cracking pressure. See the how-to in [Chapter 6](#).
- **Controlled (PID) wastegate** — an ordinary valve driven by a PID against a boost setpoint. Use it when the boost target varies with operating point, or when you want electronic control.

Chapter 6

Control systems

Control in OpenWAM is a chain: one or more **sensors** feed a **controller** (PID, 1D table, or switch), whose output drives a **controlled element**. The basics are in the tutorials; the rules below are the ones that cause confusion.

6.1 Wiring rules

There is no green line from a controller to the element it drives. WAMer draws green logic lines between linked sensor, 1D-table, and PID components, but *not* from a controller to its target. Instead a red “C” appears on the controlled component. Figure 2.3 shows both cues in one model: the green chain and the red “C” on the wastegate.

- **A sensor cannot drive an element directly.** Its output must feed a PID, 1D table, or switch; those three can drive an element directly (drag their output onto it).
- **Easiest sensor workflow:** place a dedicated *Sensor* component from the Control tab, drag from the monitored element into the sensor’s input, and configure it in the sensor’s own data panel (Figure 6.1) — simpler than digging into the element’s built-in sensor menu.
- **One controller drives many elements with the same value.** To drive different fields, attach *multiple* controllers; each picks its target field from the element’s **Control** tab drop-down.

6.2 What can be controlled

Turbine (rack position), Turbocharger (shaft speed), 4T/2T engine (engine speed, injection pressure), Valve (opening angle, lift, duration), Throttle (lift), and Cylinder (fuel mass). Controlling valve timing and lift is what makes variable-valve strategies (VANOS-style) possible.

6.3 The PID controller

The PID (Figure 6.2) has separate positive and negative gains, an initial and a constant output, output limits, a sample period, and a setpoint; its output drives whichever field is selected on the target’s Control tab. When the demanded output reaches its maximum or minimum limit, the controller stops growing the integral term so it cannot accumulate a correction it is unable to apply (anti-windup). Note that although the derivative term differentiates every solver step, a sensor that refreshes only on its own (slower) period feeds it a piecewise-constant signal, so

Property	Value
Input data	Speed
Time constant (s)	0
Gain	1
Sensor position (m)	0

Note: Time constant

OK Cancel

Figure 6.1. A sensor: measured quantity, time constant, gain, and position. Gain 0 silently disables it.

the derivative contributes little in practice — tune with P and I first. The *Glossary*, entry *PID*, lists every field.

6.4 The 1D table

A **1D Table** maps an input to an output by interpolation — linear, Hermite, or steps — refreshed on its period (Figure 6.3). Enter the points by hand, or tick *read from file* and give a .T1D filename. The file is two whitespace-separated columns, *X* then *Y*, one point per line, with no header or count. Use a 1D table for a setpoint schedule (for example, engine-speed-to-boost) or as a direct open-loop actuator.

6.5 The mechanical wastegate

The mechanical wastegate is a self-actuating spring–mass–damper valve (no controller). A diaphragm senses the boost pressure of a chosen control duct and opens the valve against a spring preload; the exhaust pressure also acts on the valve plate. The cracking boost is approximately $P_{\text{capsule}} + \text{preload}/A_{\text{diaphragm}}$; the spring constant then shapes how far it opens.

The wastegate’s force-balance fields are in internal units, not the UI labels. Pressures are in bar, areas in m², lift in m, so the spring, preload, mass, and damping are small numbers in those units, not the SI values the labels imply. Set the cracking boost with the capsule pressure and preload, then verify the saved .WAM against the validated set in the *Glossary*, entry *Wastegate*. A zero mass now raises a clear error rather than crashing.

6.6 External co-simulation does not work

The MATLAB and external-calculation nodes are non-functional — do not use them. The MATLAB-logo nodes (*External Extreme Calculation*, *Node External Calculation*) and the engine “ECU” external-MATLAB option are disabled in this build. For external logic, use the built-in controllers (PID, 1D table, switch) above, or the compiled-DLL calculation path.

Property	Value
P-parameter positive	15
I-parameter positive	60
D-parameter positive	0
P-parameter negative	15
I-parameter negative	60
D-parameter negative	0
Initial output	50
Constant output	0
Maximum output	100
Minimum output	0
Period	0.02

Note: I-parameter for positive errors

Data Controlled elements

OK Cancel

Figure 6.2. The PID controller: positive/negative gains, output limits, period, and setpoint.

Property	Data
Period	0
Interpolation mode	Hermite
Read data from file	No
Filename	

Note: Interpolation mode

Data Table Controlled elements

OK Cancel

Figure 6.3. The 1D table: period, interpolation mode, and either inline data or a .T1D file.

Chapter 7

Choosing what the solver writes

7.1 Average and instantaneous results

Each element offers two result blocks, selected by an integer case id in the element's results tab:

- **Average** — per-cycle scalars, written to AVG.DAT (the dyno numbers: torque, BMEP, BSFC, volumetric efficiency, and so on).
- **Instantaneous** — per-step traces over the last cycle, written to INS.DAT.

The complete catalogue of selectable quantities — every torque and mean-effective-pressure definition, every cylinder and duct and turbomachinery quantity — is in the *Glossary*, section *Results magnitudes*. It is the list to consult when choosing outputs; this guide does not reproduce it.

7.2 Run duration

For engine models the Execution **duration** is a number of engine *cycles*; for engine-less rigs it is *seconds*. Instantaneous results cover the last cycle only. Turbocharged and two-stroke models need many cycles to reach their limit cycle (Chapter 10).

7.3 Space-time (wave) results

To export $x-t$ wave maps of a duct, two selections are both required: tick the element's space-time box (right-click the pipe → space-time results), *and* pick the magnitudes on the Execution node (right-click → space-time results → choose Pressure/Temperature/Velocity/Mass-flow/Species). Ticking only the element boxes produces nothing. Output is one tab-separated file per magnitude, with a labelled header row; pick *pipes*, since a pipe expands to one column per cell (the spatial axis). See the *Glossary*, entry *Space-time*.

7.4 Parametric study (batch sweep)

File → Create Parametric Study generates one .WAM per case plus a .gro batch file (Figures 7.1 and 7.2). Flag the swept fields on each component, set the number of cases, and fill each case's values by hand.

Enter case values by hand. The .DAT/curve auto-import softlocks WAMer on every format — do not use it. Also, do not sweep a model that contains a mechanical wastegate: regeneration zeroes its mass field (the solver now guards this with a clear error). Each case is an ordinary run, so an unstable parameter combination can still diverge — check each case's output.

Creating parametric study...

Number of cases: 6

Element	Number	Property	Actual value	Include	Case number1	Case number2	Case number3	Case number4	Case number5	Case number6
Turbine stator	2	Inlet discharge coefficient	0.3 (Stator Coef)	Yes	0.3	0.4	0.5	0.3	0.3	0.4
		Outlet discharge coefficient	0.3 (Stator Coef)	Yes	0.3	0.4	0.5	0.3	0.3	0.4
		Reference diameter (mm)	25 (Stator Diameter)	Yes	25	25	25	25	25	25
Turbine rotor	3	Inlet discharge coefficient	0.25 (Rotor Coef)	Yes	0.25	0.25	0.25	0.35	0.45	0.35
		Outlet discharge coefficient	0.25 (Rotor Coef)	Yes	0.25	0.25	0.25	0.35	0.45	0.35
		Reference diameter (mm)	35 (Rotor Diameter)	Yes	35	35	35	35	35	35

Element: All | Property: All | Case label: All | Case number: All | Assistant selection: cte

Value: Actual value | Apply

Figure 7.1. Create Parametric Study: the per-case value grid.

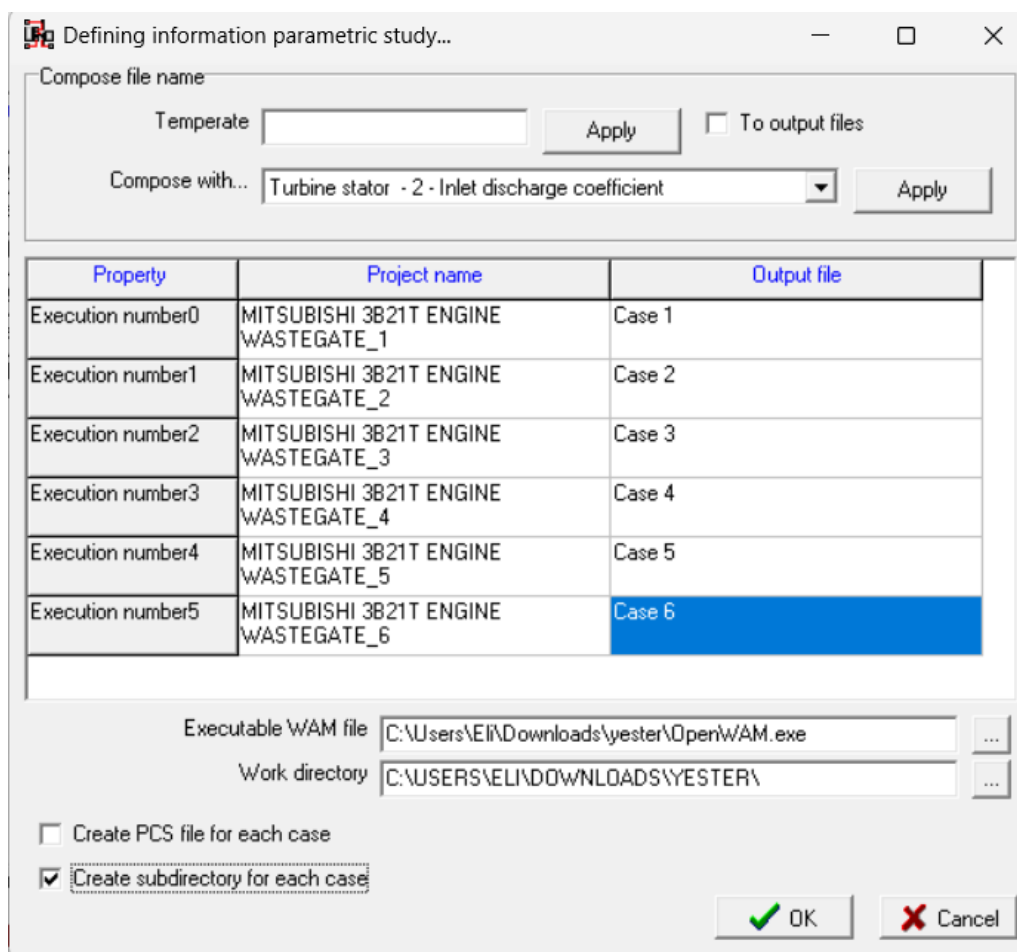


Figure 7.2. Naming the per-case project and output files, and the batch to run.

Chapter 8

Running a simulation

The plain blue play button does not run a simulation. It shows a red “X” and never produces results. Use the button to its right — the stack of papers with a play symbol — or View → Executions control.

8.1 The correct flow

1. Open the **Execution Control** window (the stack-of-papers button, or View → Executions control).
2. Click the black “+” near the left edge to add a simulation.
3. In the dialog, select the compiled `OpenWAM.exe`, the `.WAM` file, and optionally an output base name (it defaults to the `.WAM`’s directory).
4. Run. Click a running (blue-highlighted) simulation to see a live progress readout, which may take a few seconds to appear.

Status icons: a red “X” means an error; a red “?” means finished; a blue hourglass means running; red pause bars mean queued.

8.2 Other ways to run

- From PowerShell: `OpenWAM.exe my_engine.WAM`.
- Right-click the `.WAM` → Open with → `OpenWAM.exe`.

8.3 Batch runs (.gro)

In Execution Control, the folder icon opens a `.gro` file, loading a whole case list to re-run, re-order, or inspect. The save icon bundles the current queue into a `.gro` — and the queue may mix runs from different `.WAM` files.

Chapter 9

Viewing and post-processing results

9.1 The quickest path

Right-click a .DAT file → Open with → Excel (choose “always” to make double-clicking open it). The columns carry labelled, tab-separated headers, so they tabulate directly.

9.2 WAMer’s plotter

File → “open file with instantaneous and mean results” opens a .DAT in the results-visualization window (Figure 9.1). Pick an X and a Y variable, press the down-arrow to add the curve, and repeat; curves may come from different .DAT files, loaded through a second folder control. The graph icon then opens a plot window with zoom, pan, and print, and lets you save or load the axis configuration (.AXI) and the graph itself (.GRW). Figure 9.2 is a plot produced this way.

9.3 Building a trimmed results file

The two-documents icon in the results-visualization window opens the results-file-creation menu. There you can cherry-pick individual parameters (BMEP, BSFC, and so on) from one or several .DAT files into a container and save a trimmed .DAT. Save the parameter set as a template (.PLT) to reuse it without re-adding parameters one by one.

9.4 An empty graph window

The always-visible graph icon at the top opens an empty plot window, into which you can load a saved .AXI or .GRW directly.

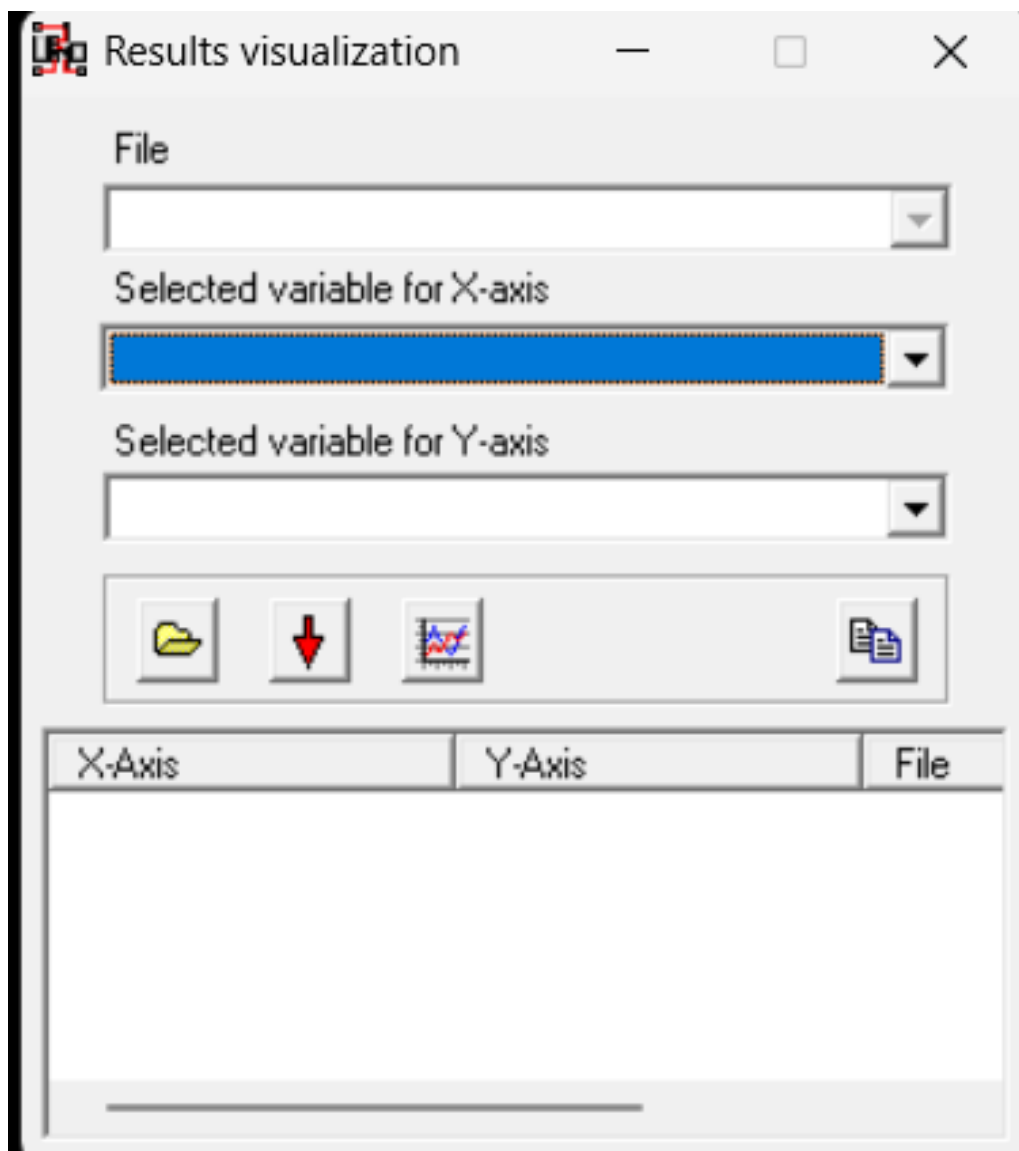


Figure 9.1. The results-visualization window: the X/Y variable pickers, the add-curve arrow, and the multi-.DAT controls.

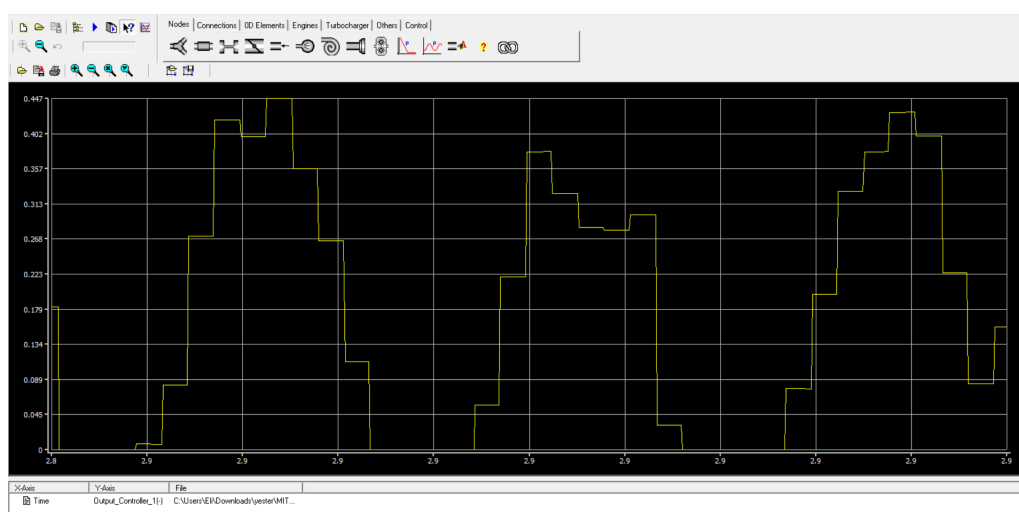


Figure 9.2. A plot from the WAMer plotter: wastegate position against time for the 3B21T at 3250 rpm, hunting up and down and damping toward its steady opening as the run converges.

Chapter 10

Stability, errors, and troubleshooting

10.1 Instability triggers, by solver method

- **Initial velocity zero** — not a trigger on the default TVD/RoeM solver. The old warning was a Lax–Wendroff-era artifact.
- **In-pipe area variation (multiple diameters)** — not a trigger on TVD/RoeM, validated stable from subsonic through supersonic (to Mach 1.39). On Lax–Wendroff+FCT it is CFL-limited and blows up above Courant ≈ 0.85 at area changes — use Courant ≤ 0.8 , or switch to TVD/RoeM.
- **Hot, rich exhaust going CFL** — lower the Courant number or raise the in-cylinder heat loss.

A diagnostic caution: a NaN at one position in a .DAT does not always mean the solver diverged — the interior can stay physical while a derived output array reports a NaN. Confirm on a TVD/RoeM run before assuming a solver bug.

10.2 Solver methods

TVD and Lax–Wendroff+FCT both work. **CE-SE is selectable in the UI but unimplemented and crashes** — do not choose it.

10.3 Crash checklist

The most common silent or cryptic failures, now mostly caught with clearer errors:

- an empty throttle Cd table, or an empty interpolation/Hermite table;
- a zero wastegate mass;
- an inlet reservoir left at 0 bar / 0°C (draws from vacuum);
- a duct output requested at exactly $x = 0$ during a hot blowdown — read at $x = 0.01$ instead.

10.4 Duration and convergence

Turbocharged and two-stroke models reach their limit cycle only after many cycles. A single controlled variable makes this concrete: Figure 9.2 shows a wastegate swinging up and down with the swings shrinking until it settles. An under-converged run reports false steady numbers, so always confirm a quantity has flattened before trusting it.

Chapter 11

Reference and cross-links

11.1 Companion documents

- *OpenWAM Glossary* — every component and every field, its meaning and a sensible default, grouped by palette tab, plus the complete results-magnitude catalogue.
- *OpenWAM Developer Guide* — each behaviour mapped to the source file that implements it.

11.2 The original tutorials

These remain useful for the material noted; where they differ from this guide, this guide is current.

- J. M. Pérez López's thesis ([Original Tutorials/JOSE-MARIA-Thesis.pdf](#)) — background theory and a full end-to-end overview. Its result-launching and viewing procedures are outdated (Chapters 8 and 9).
- Exemple d'utilisation ([Original Tutorials/Exemple-dutilisation-EN.pdf](#)) — the basic UI walkthrough and a naturally aspirated build.
- Turbocompresseur ([Original Tutorials/Turbocompresseur-EN.pdf](#)) — the basic turbocharger build and map entry.
- Lancement de simulation ([Original Tutorials/Lancement-de-simulation-EN.pdf](#)) — launching a run; superseded by Chapter 8.

11.3 File types

Extension	Contents
.WAM	the model (text) — what the solver reads
.PCS	WAMer project (binary) — editor-only, includes canvas layout and parametric studies
.DAT	results (tab-separated, labelled header)
.LEV	poppet-valve lift table
.FLU	poppet-valve flow-coefficient table
.CMP / .TMP	compressor / turbine map
.CD / .CL	rotary-disk / reed-valve discharge coefficients
.LUM	port discharge coefficients (stored in metres)
.T1D	1D-table data (two columns, X Y)
.gro	batch run list
.AXI / .GRW / .PLT	plot axes / saved graph / results-file template

A sample of each ships in the repository.